CSS 262: Linux Administration

Logging, Auditing & Cron

Lecture 11: Visibility Into Your System

📋 Today's Agenda

Part 1: systemd Journal

- `journalctl` filtering & options
- Boot logs, unit logs, priority filtering
- Log persistence across reboots

Part 2: syslog & rsyslog

- Traditional syslog architecture
- Facilities and severities
- rsyslog rules and remote logging

Part 3: logrotate & auditd

- Preventing disk fill with logrotate
- `auditd` : kernel-level audit framework
- Writing watch rules, `ausearch` , `aureport`

Part 4: Cron & Scheduled Tasks

- crontab syntax and fields
- System-wide cron directories
- `at` for one-off jobs, `systemd` timers

🔄 Quick Recap: Week 10

SELinux

- DAC (file permissions) vs MAC (policy)
- Modes: enforcing, permissive, disabled
- Security contexts: `user:role:type:level`
- Booleans: `setsebool -P`
- Troubleshooting: `ausearch` → `audit2why`

Key Takeaways

- SELinux confines **what** processes can do
- But who's watching **what** they do?
- An intruder may already be inside – logs are your evidence
- **Today**: Build visibility – detect, record, schedule

🔍 **You can't defend what you can't see.** Logging and auditing are the foundation of incident detection and forensics.

Part 1

📓 systemd Journal (`journalctl`)

📚 systemd Journal Overview

What It Is

- Binary log store managed by `systemd-journald`
- Collects kernel messages, syslog, stdout/stderr of units
- Stored in `/run/log/journal/` (volatile) or `/var/log/journal/` (persistent)
- Indexed – fast filtering by unit, time, priority, field

Why It Matters

- One place for all system and service logs
- Survives rapid log rotation (binary format)
- Correlated with systemd unit lifecycle events
- Works without syslogd – but can forward to it

💡 **Persistence:** By default the journal is volatile (lost on reboot). Create `/var/log/journal/` or set `Storage=persistent` in `/etc/systemd/journald.conf` to persist it.

🔍 `journalctl` – Common Usage

```
# Follow new messages in real time (like tail -f)
journalctl -f

# Show messages for a specific unit
journalctl -u sshd
journalctl -u nginx -f

# Show messages since boot
journalctl -b
journalctl -b -1          # Previous boot

# Filter by time
journalctl --since "2024-01-15 10:00:00"
journalctl --since "1 hour ago"
journalctl --since today

# Filter by priority (emerg=0 to debug=7)
journalctl -p err           # error and above
journalctl -p warning..err  # range

# Show kernel messages only
journalctl -k
```

🔍 `journalctl` – More Filters

```
# Show last N lines
journalctl -n 50

# No pager (pipe-friendly)
journalctl --no-pager

# JSON output (for parsing)
journalctl -o json-pretty -n 5

# Filter by executable or PID
journalctl /usr/sbin/sshd
journalctl _PID=1234

# Combine filters
journalctl -u sshd --since today -p err

# Disk usage / cleanup
journalctl --disk-usage
journalctl --vacuum-size=500M
```

✅ **For security investigations:** `journalctl -u sshd --since yesterday | grep "Failed"` – find all failed SSH logins.

📊 Log Priorities (Syslog Severity Levels)

Number	Name	Meaning	Example
0	**emerg**	System unusable	Kernel panic
1	**alert**	Immediate action needed	Corrupted disk
2	**crit**	Critical conditions	Hardware failure
3	**err**	Error conditions	Service start failure
4	**warning**	Warning conditions	Config deprecated
5	**notice**	Normal but significant	Service started
6	**info**	Informational	User logged in
7	**debug**	Debug-level messages	Verbose tracing

💡 `journalctl -p err` shows **err and higher severity** (0–3). Lower number = higher severity.

⚙️ Configuring journald

`/etc/systemd/journald.conf`

```
[Journal]
Storage=persistent      # volatile | persistent | auto | none
Compress=yes            # Compress stored entries
SystemMaxUse=500M       # Max disk space for system journal
RuntimeMaxUse=100M      # Max for runtime (volatile) journal
MaxRetentionSec=1month  # Discard entries older than this
ForwardToSyslog=yes     # Forward to rsyslog/syslogd
```

```
# Apply changes
sudo systemctl restart systemd-journald

# Make journal persistent now (without restarting)
sudo mkdir -p /var/log/journal
sudo systemd-tmpfiles --create --prefix /var/log/journal
```

⚠️ **For compliance and forensics:** Always configure persistent journal storage. Volatile logs vanish on reboot – useless for incident response.

Part 2

📜 syslog & rsyslog

📜 Traditional syslog Architecture

How It Works

- Applications log via the `syslog()` syscall
- The syslog daemon receives messages and routes them
- Routing based on **facility** (what's logging) + **severity** (how bad)
- Writes to files in `/var/log/`
- Can forward to remote log servers

Key Log Files

```
/var/log/syslog      # General log (Debian/Ubuntu)
/var/log/messages    # General log (RHEL/CentOS)
/var/log/auth.log    # Auth events (Debian/Ubuntu)
/var/log/secure      # Auth events (RHEL/CentOS)
/var/log/kern.log    # Kernel messages
/var/log/cron        # Cron job output
```

💡 **rsyslog** is the most common syslog daemon on modern Linux. It is compatible with classic syslog but adds TCP forwarding, templates, and filtering.

📊 Syslog Facilities

Facility	Code	Source
`kern`	0	Kernel messages
`user`	1	User-space processes
`daemon`	3	System daemons
`auth`	4	Security/authentication
`cron`	9	Cron daemon
`local0–7`	16–23	Locally-defined

💡 Rules match `facility.severity`. Example: `auth.info` = auth at info-level and above. `*.*` = everything.

⚙️ rsyslog Configuration

`/etc/rsyslog.conf` and `/etc/rsyslog.d/*.conf`

```
# Selector syntax:  facility.severity    destination
auth,authpriv.*          /var/log/auth.log
*.*;auth,authpriv.none   -/var/log/syslog   # leading - = async write
kern.*                   /var/log/kern.log
cron.*                   /var/log/cron.log
```

Forward to Remote Syslog Server

```
# /etc/rsyslog.d/50-remote.conf

# UDP (traditional, no delivery guarantee)
*.* @192.168.1.100:514

# TCP (reliable)
*.* @@192.168.1.100:514
```

🚨 **Security best practice:** Forward to a remote log server. An attacker who compromises a host may delete local logs – remote copies survive.

🔍 Reading Logs

Traditional Text Log Commands

```
# Tail a file live
tail -f /var/log/auth.log

# Last 100 lines
tail -n 100 /var/log/syslog

# Search for a pattern
grep "Failed password" /var/log/auth.log
grep "sshd" /var/log/auth.log | grep -i "invalid"

# Count failed SSH logins per IP
grep "Failed password" /var/log/auth.log \
  | awk '{print $11}' | sort | uniq -c | sort -rn
```

✅ **Quick SSH investigation:** The above pipeline shows which IPs are brute-forcing your server and how many attempts each made.

💡 On systemd systems, `journalctl -u sshd` is often more convenient than grepping text logs — it's indexed and faster.

Part 3

🔄 logrotate & auditd

🔄 logrotate – Preventing Disk Fill

The Problem

- Log files grow unbounded without rotation
- A busy web server: access.log can hit gigabytes/day
- Full disk → service crashes, denial of service
- Old logs consume space that's no longer needed

How logrotate Works

- Runs daily via cron (or systemd timer)
- Renames old log, creates new empty log
- Compresses old files, deletes after N rotations
- Sends `SIGHUP` to services to reopen logs
- Config: `/etc/logrotate.conf` + `/etc/logrotate.d/`

💡 **Result:** `/var/log/syslog`, `/var/log/syslog.1`, `/var/log/syslog.2.gz`, … – controlled rotation window.

⚙️ logrotate Configuration

`/etc/logrotate.d/myapp`

```
/var/log/myapp/*.log {
    daily                # Rotate daily
    rotate 14            # Keep 14 rotated files
    compress             # gzip old files
    delaycompress        # Don't compress the most recent rotation
    missingok            # Don't error if log file is missing
    notifempty           # Don't rotate empty files
    create 0640 www-data adm  # Create new log with these perms
    sharedscripts        # Run postrotate once for all matched files
    postrotate
        # Tell the service to reopen its log file
        /bin/kill -HUP $(cat /var/run/myapp.pid) 2>/dev/null || true
    endscript
}
```

Test Without Actually Rotating

```
sudo logrotate -d /etc/logrotate.d/myapp     # Debug/dry run
sudo logrotate -f /etc/logrotate.d/myapp     # Force rotation
```

🔍 auditd – Kernel Audit Framework

What auditd Does

- Kernel-level audit subsystem (`kauditd`)
- Records: file access, syscalls, logins, privilege use
- Writes to `/var/log/audit/audit.log`
- Tamper-resistant: requires root to stop, kernel-driven
- Used by SELinux, Fail2Ban, compliance frameworks

Why It's Important

- Detects access to sensitive files
- Records who ran what with `sudo`
- Required by PCI-DSS, HIPAA, DISA STIG
- Forensic evidence after an incident
- Captures events even text logs don't

🔑 **auditd vs syslog:** syslog is application-driven (apps decide what to log). auditd is kernel-driven (the kernel records events regardless of the application).

⚙️ auditd: Install & Basic Config

```
# Install (Debian/Ubuntu)
sudo apt install auditd audispd-plugins

# Install (RHEL/CentOS)
sudo yum install audit

# Enable and start
sudo systemctl enable --now auditd

# Status
sudo systemctl status auditd
sudo auditctl -s        # Kernel audit status
sudo auditctl -l        # List loaded rules
```

Configuration Files

```
/etc/audit/auditd.conf      # Daemon settings (log path, size, rotation)
/etc/audit/audit.rules      # Persistent rules (loaded on start)
/etc/audit/rules.d/*.rules  # Rules in drop-in directory (preferred)
```

💡 Rules added with `auditctl` are runtime-only. Use `/etc/audit/rules.d/` for persistence. Run `augenrules --load` to reload.

📝 Writing Audit Rules

`auditctl` Rule Syntax

```
# Watch a file (-w path, -p permissions, -k tag)
sudo auditctl -w /etc/passwd -p rwa -k passwd_changes

# Watch a directory recursively
sudo auditctl -w /etc/ssh/ -p rwxa -k ssh_config

# Audit a syscall (always,exit = record on syscall return)
sudo auditctl -a always,exit -F arch=b64 -S execve -k exec_commands
```

📝 Common Audit Rule Sets

```
# /etc/audit/rules.d/hardening.rules

# Watch for changes to passwd and shadow
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group  -p wa -k identity
-w /etc/gshadow -p wa -k identity

# Watch SSH configuration
-w /etc/ssh/sshd_config -p wa -k sshd_config

# Log all sudo commands
-w /usr/bin/sudo -p x -k sudo_usage

# Log failed login attempts
-w /var/log/faillog -p wa -k logins
-w /var/log/lastlog -p wa -k logins

# Detect privilege escalation attempts
-a always,exit -F arch=b64 -S setuid -k priv_esc
```

✅ These rules align with CIS Benchmark for Linux and DISA STIG recommendations.

🔍 `ausearch` – Querying the Audit Log

```
# Search by key tag
sudo ausearch -k passwd_changes

# Search by time range
sudo ausearch -ts today
sudo ausearch -ts "04/01/2024 10:00:00" -te "04/01/2024 11:00:00"

# Search by username
sudo ausearch -ua alice

# Search by event type
sudo ausearch -m USER_AUTH          # Authentication events
sudo ausearch -m USER_CMD           # sudo commands
sudo ausearch -m AVC                # SELinux denials
sudo ausearch -m SYSCALL -k exec_commands

# Combined
sudo ausearch -k identity -ts today --interpret
```

💡 `--interpret` flag translates numeric UIDs and syscall numbers to human-readable names.

📊 `aureport` – Summary Reports

```
# Summary of all events
sudo aureport

# Authentication events
sudo aureport --auth

# Failed authentication
sudo aureport --auth --failed

# Summary of executable events
sudo aureport --executable

# List of all users who ran commands
sudo aureport --user

# Events in a time range
sudo aureport --start today --end now

# Commands run via sudo
sudo aureport --comm --success
```

🔑 **aureport is ideal for periodic reporting** (daily digest) while ausearch is for targeted investigations.

Part 4

⏰ Cron & Scheduled Tasks

⏰ What is Cron?

The cron Daemon

- Runs scheduled commands at specified times
- `crond` reads crontab files on startup and after changes
- Per-user crontabs: `crontab -e`
- System crontabs: `/etc/crontab`, `/etc/cron.d/`
- Logs output to syslog (facility `cron`)

Why It Matters for Security

- Attackers install cron persistence for re-entry
- Scheduled tasks run as specific users – privilege concern
- Cron output is often silently discarded (MAILTO)
- Malicious crons are a common persistence mechanism
- Must audit crontabs as part of incident response

🚨 **Incident response:** After a breach, always check `crontab -l` for all users and `/etc/cron.d/` for attacker persistence.

📝 Crontab Syntax

```
┌───────────── minute       (0 - 59)
│ ┌─────────── hour         (0 - 23)
│ │ ┌───────── day of month (1 - 31)
│ │ │ ┌─────── month        (1 - 12 or Jan-Dec)
│ │ │ │ ┌───── day of week  (0 - 7, Sunday = 0 or 7)
│ │ │ │ │
* * * * *  command to execute
```

Examples

```
# Every minute
* * * * *   /usr/bin/check-disk.sh

# Daily at 2:30am
30 2 * * *  /usr/bin/backup.sh

# Weekdays at 8am
0 8 * * 1-5  /usr/bin/send-report.sh

# Every 15 minutes
*/15 * * * *  /usr/bin/poll-api.sh
```

⚙️ Managing Crontabs

User Crontabs

```
crontab -e                # Edit own crontab
crontab -l                # List own crontab
crontab -r                # Remove own crontab
sudo crontab -u alice -e  # Edit alice's crontab
sudo crontab -u alice -l  # List alice's crontab
```

System Cron Locations

```
/etc/crontab       # System crontab (+ user field)
/etc/cron.d/       # Drop-in files
/etc/cron.hourly/  # Run hourly
/etc/cron.daily/   # Run daily
/etc/cron.weekly/  # Run weekly
/etc/cron.monthly/ # Run monthly
```

💡 Scripts in `/etc/cron.daily/` etc. are executed by `run-parts` – must be executable and have no dots in the filename.

System Crontab Format

`/etc/crontab` – Has a User Field

```
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# min hour dom month dow  user    command
17  *  * * *  root  cd / && run-parts --report /etc/cron.hourly
25  6  * * *  root  test -x /usr/sbin/anacron || run-parts /etc/cron.daily
47  6  * * 7  root  test -x /usr/sbin/anacron || run-parts /etc/cron.weekly
52  6  1 * *  root  test -x /usr/sbin/anacron || run-parts /etc/cron.monthly
```

Drop-in Files: `/etc/cron.d/`

```
# /etc/cron.d/myapp
MAILTO=admin@example.com

0 3 * * *  appuser  /opt/myapp/bin/cleanup.sh >> /var/log/myapp-cleanup.log 2>&1
```

🔑 **User field:** System crontabs specify which user runs the command. Per-user crontabs (via `crontab -e`) do not have this field.

⏰ Cron Output & MAILTO

```
# Capture output in the crontab itself
*/5 * * * *  /usr/bin/check.sh >> /var/log/check.log 2>&1

# Email output to a specific address
MAILTO=admin@example.com
0 2 * * *  /usr/bin/backup.sh

# Discard all output (silent mode — use carefully)
0 2 * * *  /usr/bin/backup.sh > /dev/null 2>&1
```

🚨 **Don't silently discard cron output in production.** If a job fails, you'll never know. Log to a file or set `MAILTO`.

💡 Default behavior without redirection: cron emails output to the running user via local mail. On servers without mail configured, this silently disappears.

🕐 `at` – One-Off Scheduled Jobs

```
# Install
sudo apt install at

# Schedule a job for a specific time
echo "/usr/bin/backup.sh" | at 02:30
echo "/usr/bin/reboot" | at 23:00 + 1 day

# Interactive input
at now + 5 minutes
at> /usr/bin/cleanup.sh
at>
(press Ctrl+D to submit)

# List pending at jobs
atq

# Remove a pending job
atrm 3           # Remove job #3

# View a job's content
at -c 3          # Show job #3's commands
```

💡 `at` is useful for one-time deferred tasks (run this script after office hours). Cron is for recurring tasks.

⏱️ systemd Timers

A `.timer` unit triggers a `.service` unit — output goes to the journal, no silent failures.

💡 **Advantages over cron:** Journal integration, `Persistent=true` handles missed runs, systemd dependency management.

```
# /etc/systemd/system/backup.timer
[Timer]
OnCalendar=daily
Persistent=true
Unit=backup.service
[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now backup.timer
systemctl list-timers --all
```

🔒 Cron Security Considerations

Controlling Who Can Use Cron

```
# Allow only listed users (if file exists)
/etc/cron.allow

# Block listed users (if cron.allow doesn't exist)
/etc/cron.deny
```

Audit All Crontabs

```
# Check all user crontabs
for user in $(cut -d: -f1 /etc/passwd); do
    entries=$(crontab -u $user -l 2>/dev/null)
    [ -n "$entries" ] && echo "═══ $user ═══" && echo "$entries"
done

# Check system cron directories
ls -la /etc/cron.d/ /etc/cron.daily/ /etc/cron.hourly/
cat /etc/crontab
```

🚨 **Attacker persistence:** A cron job running as root every minute is a classic backdoor. Audit all crontabs regularly and after any security incident.

📊 Putting It Together: Detection Workflow

Scenario: Detecting a Brute-Force SSH Attack

```
# 1. Check for repeated failed logins (journald)
journalctl -u sshd --since today | grep "Failed password" | head -20

# 2. Count top attacking IPs
journalctl -u sshd --since today --no-pager \
  | grep "Failed password" \
  | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -10

# 3. Check if any succeeded after failures
journalctl -u sshd --since today | grep "Accepted"

# 4. Correlate with audit log (were any files accessed?)
sudo ausearch -ua suspicioususer -ts today --interpret

# 5. Check for new cron persistence
sudo crontab -u suspicioususer -l 2>/dev/null
ls -lt /etc/cron.d/      # Recently modified?
```

✅ Logging & Auditing Checklist

Logging

- ☐ Journal storage is persistent (`/var/log/journal/`)
- ☐ `journald.conf` retention configured
- ☐ rsyslog forwarding to remote log server
- ☐ `/var/log/auth.log` or `/var/log/secure` monitored
- ☐ logrotate configured for all app logs

Auditing & Cron

- ☐ `auditd` installed and enabled
- ☐ Audit rules for `/etc/passwd` , `/etc/shadow` , `/etc/ssh/`
- ☐ `MAILTO` set for cron jobs or output redirected
- ☐ Crontabs audited for all users
- ☐ `/etc/cron.d/` contents reviewed
- ☐ `at` queue empty or expected

📊 Summary: Logging, Auditing & Cron

Key Concepts

1. **journald**: Indexed binary log, `journalctl` for filtering
2. **rsyslog**: Text-based, facility.severity routing, remote forwarding
3. **logrotate**: Prevent disk fill, compress and rotate on schedule
4. **auditd**: Kernel-driven audit, watch rules, `ausearch` / `aureport`
5. **cron**: Recurring scheduled tasks, five time fields + command
6. **at**: One-off deferred jobs
7. **systemd timers**: Modern cron with journal integration

Key Files

File/Command	Purpose
`/var/log/journal/`	Persistent journal store
`/etc/rsyslog.d/`	rsyslog rules
`/etc/logrotate.d/`	Log rotation config
`/var/log/audit/audit.log`	auditd log
`/etc/audit/rules.d/`	Persistent audit rules
`crontab -e`	Edit user crontab
`/etc/cron.d/`	System drop-in crons

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Use `journalctl` to filter logs by unit, time, priority, and keyword
- ✅ Configure persistent journal storage
- ✅ Explain syslog facility and severity and read rsyslog rules
- ✅ Configure logrotate to rotate application logs
- ✅ Install and configure `auditd` with watch rules
- ✅ Query audit events with `ausearch` and generate reports with `aureport`
- ✅ Write crontab entries using the five-field time syntax
- ✅ Distinguish per-user crontabs from system cron directories
- ✅ Schedule one-off jobs with `at`
- ✅ Audit crontabs as part of incident response

🎓 **Next Week:** Containerization (Docker) — namespaces, cgroups, images, and container security!

🧪 Lab Practice: Auditd & Logrotate

Exercise 1: journalctl Investigation

Filter `journalctl` to show only SSH-related events since yesterday at priority `info` and above. Find any failed authentication attempts. Count unique source IPs.

Exercise 2: logrotate Configuration

Write a logrotate config for `/var/log/myapp/*.log`: rotate daily, keep 7 files, compress, create with permissions 0640. Test with `logrotate -d`.

Exercise 3: Audit Rules

Install `auditd`. Add rules to watch `/etc/passwd` and `/etc/sudoers` for writes. Edit `/etc/passwd` (e.g. change a comment field). Use `ausearch -k` to find the event. Run `aureport --auth`.

Exercise 4: Cron Scheduling

Create a crontab entry that appends a timestamp to `/tmp/heartbeat.log` every 5 minutes. After 15 minutes, verify the log has three entries. Then disable the job.

🔗 Additional Resources

Documentation

- systemd journal – Official journalctl manual
- rsyslog Documentation – Configuration reference
- Linux Audit Documentation – Red Hat audit guide
- logrotate man page

Books & Guides

- *"The Linux Command Line"* by William Shotts – Chapters on shell and text processing
- NIST SP 800-92 – Guide to Computer Security Log Management
- CIS Benchmark for Linux – Logging and auditing section

Practice

- Set up auditd, trigger events, and write a short incident report
- Configure rsyslog forwarding to a second VM and verify logs arrive
- Write a `systemd` timer that replaces an existing cron job

Questions?

Next: Containerization (Docker)